RESEARCH REPORT / SEPTEMBER 2026

Synthetic coding RL environments

Papers, public implementations, verifier design, and a practical direction for Repo2RLEnv.

The engineering question	The research question
Can this PR become a faithful, reproducible task with trustworthy grading?	Can accepted tasks be expanded into a diverse curriculum that keeps improving a learner?

Includes SWE-rebench, TaskPilot, Environment Evolution for Terminal Agents, PR builders, repository synthesis, terminal generation, self-play and adversarial verification.

Checked 10 September 2026. Papers and source code were inspected; target generators and cloud jobs were not executed. Release findings distinguish complete source, partial components, datasets and descriptions.

Reading guide

Start with chapters 1-2 for the conclusions, 3-7 for the landscape, and 8-11 for quality, yield, implementation and source-level reuse. Every work has direct primary-source links.

1. What the landscape says

The components of Repo2RLEnv already exist across several projects. The opportunity is to integrate them around a stronger, auditable acceptance contract. No single inspected system establishes automatic, high-quality conversion of every arbitrary PR. The closest public engineering baseline is SWE-gen; RepoLaunch and SWE-Next offer particularly relevant approaches to amortizing repository setup. Adaptive synthesis adds a second capability: expanding useful seeds into a continuing supply of training tasks.

The practical recommendation is to build a faithful PR converter first, maintain explicit evidence of verifier quality, and evolve accepted tasks separately. A good environment can be easy for Opus and still be valuable for another learner. Conversely, a hard task can be ambiguous, broken or exploitable. Correctness, difficulty and learning value need separate measurements.

Priority	Reference to inspect	What to reuse
PR packaging	SWE-gen	PR-to-Harbor construction, NOP/oracle runs, trace classification
Repository setup	RepoLaunch, SWE-Next	Reusable images, commands, dependency profiles and parsers
Missing tests	THUDM SWE-Dev, R2E	Behavioral test generation and execution-guided refinement
Verifier challenges	SWE-smith, SWE-Mutation, harden-v0	Plausible wrong patches, adversarial attacks, legitimate-solution preservation
Terminal synthesis	TerminalWorld, SETA, Endless Terminals	Executable authoring loops, environment reconstruction and evolution
Adaptive curriculum	CalibForge, TaskPilot, Environment Evolution	Trace-directed revision and ordered task families; generators not located

These are recommendations from source inspection, not results of running the repositories. Source availability is classified throughout as: **generator present**, **partial components**, **data/evaluation release**, or **official implementation not located**. Public code with no explicit license is identified separately. Dataset, code and upstream-repository licenses can differ.

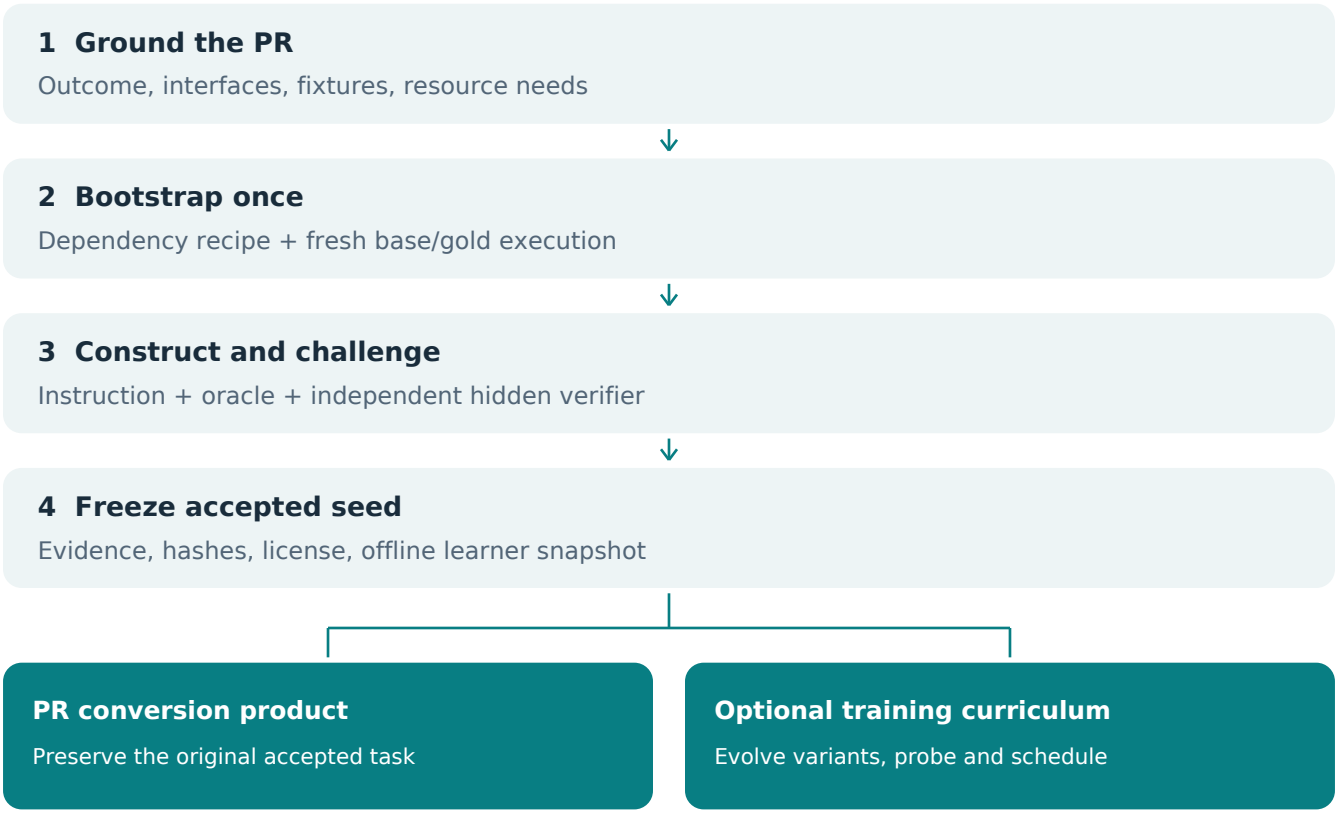
Three findings should change our implementation priorities. First, build reuse is an independent product: a verifier failure should not discard a healthy dependency image. Second, an author producing both a patch and tests can accidentally encode the same mistake twice. Third, high reported yields often start after repositories, specifications or seeds have already been filtered. Comparing those percentages to end-to-end PR acceptance is misleading.

2. A map of the approaches

Family	Starting material and generated object	Principal limitation
PR or commit mining	Human change -> reproducible issue, image and grading harness	Historical setup; tests may underspecify the intended change
Repository synthesis	Working code -> mutation, feature reconstruction or transferred issue	Artificial faults and reference-correlated tests
Terminal synthesis	Skills, documents or recordings -> workspace, goal and verifier	Invented requirements, brittle construction and weak grading
Adaptive evolution	Accepted environments + feedback -> revised tasks or task lineages	Difficulty can be mistaken for quality; distribution can drift
Joint self-play	Trainable proposer and solver -> evolving task distribution	Requires training infrastructure and robust generator rewards
Problem/trajectory synthesis	Code snippets or contest questions -> tests and demonstrations	Often no real repository or independently resettable environment

The word "synthetic" covers different objects. SWE-rebench largely preserves human problem/fix/test evidence while generating execution support. SWE-smith invents defects in working code. TaskPilot invents repository tasks and calibrates them against a learner. SERA synthesizes useful training trajectories without requiring a behavioral test oracle. These should be evaluated on different claims.

For Tasksmith, the following is a proposed architecture derived from the survey. The fork after acceptance is deliberate: task evolution should not overwrite or inflate the reported conversion rate of the original PR.



Proposed Tasksmith architecture; synthesis of this survey, not a reproduced paper figure.

The unit of release should be an environment **plus its evidence**: source provenance, public contract, dependency recipe, immutable runtime identifiers, verifier revision, validation runs, challenge outcomes and rollout diagnoses. A Dockerfile and a passing reference patch are necessary evidence, but do not establish the whole claim.

3. Mining real PRs and building environments

SWE-gen - the closest PR-to-Harbor baseline

SWE-gen converts merged PRs into Harbor tasks. Claude Code completes installation and verifier scaffolding; validation expects NOP reward 0 and oracle reward 1. It supports cloud backends including Modal and Daytona, and reuses successful tasks as repository-specific references. The public Apache-2.0 implementation includes construction, validation, farming and rollout classification. Its configurable selection rules prefer test-bearing, linked-issue PRs. Repository and release.

The inspected verifier checks exact reward values and retains the image between NOP and oracle runs. Its construction prompt concentrates on PR-specific tests; broad regression protection and independent verifier adequacy still need additional work. This is an excellent baseline to compare before writing more bootstrap orchestration. Validator source, construction prompt.

SWE-rebench - large-scale executable PR collection

The original paper reports 21,336 tasks across 3,468 repositories. Qwen2.5-72B-Instruct proposes and repairs environment recipes; setups are grouped by repository version, with dependency snapshots and Buildah infrastructure. Human issues, fixes and PR tests supply the core task evidence. The public dataset is substantial; the evaluation fork is not evidence of a complete production miner release. Its filtered split overlaps the main split and should not be added to the unique task count. Paper, May 2025, dataset.

SWE-rebench V2 - multilingual setup and quality filtering

V2 reports 583,809 eligible PRs from 21,692 repositories, 41,349 F2P tasks, and 32,079 tasks after issue-clarity filtering, across 20 languages. Qwen3-Coder-480B-A35B-Instruct handles setup and parser generation; separate models judge clarity. Repository setup is reused, compiled projects are rebuilt, and full-suite pre/post results are compared. Paper, February 2026.

The MIT release contains Dockerfiles, installer/parser/description prompts and build/evaluation scripts. The inspected tree did not expose the complete mining and interactive setup orchestrator. Its installer prompt is particularly useful for prioritizing CI, manifests, test collection and fresh execution. Released components, 32,079-row dataset.

SWE-Gym - executable environments required substantial curation

SWE-Gym reports 2,438 tasks from 11 selected repositories, from a much larger candidate pool. Construction involved roughly 200 human-hours and 10,000 CPU-core-hours. Human issues, fixes and tests are paired with generated solver trajectories and verifier-training data. Main code emphasizes training; the separate SWE-Bench fork contains collection, versioning and evaluation machinery. This is valuable reproducibility infrastructure, not evidence of a universal autonomous bootstrapper. Paper, December 2024, project, construction/evaluation fork.

R2E-Gym / SWEGEN - real generation, repository-specific setup

R2E-Gym mines commits, uses execution evidence to construct descriptions and can generate tests conditioned on the reference. The paper body/release reports 8,135 full tasks and 4,578 decontaminated training tasks; the abstract contains older inconsistent naming/counts. Public Apache-2.0 code now includes generation components. Adding repositories still requires enums, commands and installation handling. Description construction sees the gold diff and failing tests, so leakage review matters. Paper, April 2025, generation guide, source.

SWE-Factory - role separation and targeted repair

Four agents retrieve context, write Dockerfiles, write evaluation scripts and analyze tests. Successful setups are retrieved from nearby versions. The paper tests 671 multilingual issues; GPT-4.1-mini produces 269 valid instances under its definition. Public construction code exists. The important limitation is that the inspected F2P judge accepts nonzero-to-zero transitions without reliably distinguishing assertion failures from missing dependencies; the README recommends manual review of error-to-pass cases. Paper, June 2025, code, acceptance logic.

The license states AGPL for noncommercial use with a separate commercial license option. It should not be summarized as ordinary unrestricted AGPL. License terms.

SWE-bench-Live and RepoLaunch - strongest reusable bootstrap reference

SWE-bench-Live continuously mines issue/PR tasks; its initial release reports 1,319 tasks from 93 repositories. RepoLaunch separately automates dependency installation, rebuilding, test commands and parsers across languages and platforms. Both have MIT source. RepoLaunch uses LangGraph and exposes setup, verification and save stages. Live paper, RepoLaunch paper, Live code, RepoLaunch code.

Its memory mechanism builds a representative commit and reuses the image, commands and parser for adjacent commits. The project reports at least 98% build success on 856 issues from 93 repositories with substantial API/storage savings; this is a specific reuse experiment, not arbitrary-PR acceptance. The code uses passed-test counts to assess reuse, which does not prove semantic correctness of an individual PR. Memory documentation, adjacent-commit implementation.

SWE-Universe - large reported scale, unreleased builder

SWE-Universe reports 807,693 environments after mining and construction. A custom Qwen-Next-80A3 backbone handles patch splitting, building and hacking detection; agents can switch between base and gold states while constructing a verifier. Docker layers are cached. The feedback loop explicitly critiques source-pattern shortcuts. No official generator or dataset was located in the paper-linked and exact-name searches. The reported scale therefore cannot be treated as downloadable inventory or independently reproduced quality. Paper, February 2026.

OpenSWE / daVinci-Env - public builder at substantial reported cost

The paper reports 572,114 candidate PRs yielding 45,320 environments across about 12,800 repositories. DeepSeek-V3.2 constructs environments; GLM-4.7 collects solver trajectories. Construction uses cached repositories, stable image layers and targeted repair, retaining images when only verifier scripts change. Reported project totals include $891,000 for construction and $576,000 for curation; these are not comparable to API-only per-instance quotes elsewhere. Paper, March 2026.

Real multi-agent builder and distributed-worker source is public. The HF dataset is auto-gated; metadata was accessible, unauthenticated raw downloads were not. The derived application code retains SWE-Factory's special licensing conditions. Repository, dataset, license.

ScaleSWE - distinguish the paper scale from the release

Three agents construct setup, tests and descriptions from PR evidence. The paper reports 100,000 tasks; the project announces roughly 20,000 public tasks plus large trajectory releases. Its schema distinguishes developer F2P patches from agent-generated F2P scripts. The inspected repository contains release material and trajectories, with no generator. Code, dataset and upstream licensing need separate interpretation. Paper, February 2026, project, data.

SWE-Bench++ - useful three-state validation

This Turing project reports 11,133 tasks across 3,971 repositories and 11 languages. It distinguishes base, test-only and full-PR execution, tests parsers using injected failures, and repeats builds/gold runs. Public source includes multilingual evaluation and templates; the complete generation/AutoQA system was not located. The public HF metadata showed 500 rows, not the full paper count. Missing-symbol failures for new features require careful interpretation rather than blanket rejection. Paper, December 2025, repository, dataset.

SWE-Next - dependency profiles separate from task source

SWE-Next reports 102,582 candidate commit pairs yielding 2,308 tasks. Repository-quarter dependency profiles amortize setup; images omit repository source until composition. Full Apache-2.0 mining, profile generation, execution, description and export code is present. Its execution comparison uses common test names with a file-level fallback; "no regression" applies to the compared evidence, not every possible behavior. This is a particularly useful design for repeated PRs from the same repository. Paper, March 2026, implementation.

4. Synthesizing tasks inside repositories

SWE-smith - broad, reusable mutation machinery

SWE-smith starts from working repository images, then introduces defects using LLM edits, AST transformations, PR inversion and combinations. It retains mutations that break previously passing tests. The original paper reports 50,137 tasks from 128 repositories and 50.1% candidate-mutation yield; setup included manual verification. o3-mini generates bugs and Claude 3.7 Sonnet supplies most expert repairs. Paper, April 2025.

MIT source exposes generation, profiles and execution grading. Its strongest use for Tasksmith is not merely producing more bugs: it can supply plausible incorrect patches to challenge a verifier. Existing tests offer an oracle separate from the mutation author, but passing them does not establish complete behavior coverage. Code, validation.

BugPilot - natural regressions from attempted features

Claude Sonnet 4 in SWE-agent invents and implements features in prepared SWE-smith images; resulting regressions become repair tasks. This aims to produce more natural mistakes than deliberate bug insertion. Sonnet 4 also collects repair trajectories. Existing tests supply execution labels. The audited comparative subset contains 785 feature-addition tasks from 86 repositories. Microsoft releases FrogBoss weights, but an official BugPilot generator was not located. A key design question is whether grading also preserves the intended new feature. Paper, October 2025, official blog.

SWE-Synth - coverage-guided mutants and natural repairs

Functions/classes are masked and regenerated; uncompilable or test-passing variants are rejected. Qwen2.5-Coder-32B-Instruct generates both mutants and repair rollouts. The funnel is 21,804 variants, 9,459 verifiable bugs and 3,018 successful repair trajectories, across seven repositories. Those are three different counts. Actual mutation and Docker-validation code is public, but no repository license file was detected. The main reported training use is SFT; executable tasks can also support RL. Paper, April 2025, implementation.

SWE-Flow - test-guided feature decomposition

Runtime test traces form dependency graphs and incremental development schedules. Selected function bodies are removed; descriptions and reference restorations define reconstruction tasks. The paper reports 16,061 training tasks and 2,020 test tasks across 74 projects. Native tests provide the oracle; asynchronous/multiprocess tracing is a stated limitation. Public MIT pipeline and tracer code are available. This is useful for finding a coherent, test-supported slice of a large PR, while keeping reconstruction distinct from novel feature synthesis. Paper, June 2025, pipeline.

SWE-Dev, THUDM - the missing-test problem

For PRs without sufficient tests, this project generates Gherkin descriptions, executable tests and repairs. Its strongest mixed writer uses Llama3.1-70B-Instruct for descriptions and Qwen2.5-Coder32B for tests. From 26,000 attempted instances it obtains 2,097 F2P functions, later combined with existing tests into 4,630 test cases. These are not environment counts. MIT crawling, generation and evaluation code is public. Tasksmith should borrow the test-construction process while distinguishing dependency errors from intended missing behavior. Paper, June 2025, code.

SWE-Dev, feature-driven - a different project

The separate feature-driven SWE-Dev paper uses test call trees to choose core functions, masks implementations and refines docstrings into requirements with GPT-4o. It reports 14,000 training and 500 test samples, with SFT/RL experiments. Its advertised repository returned 404 during this review. Do not substitute THUDM's repository as its implementation. Paper, May 2025, advertised repository.

R2E - reference execution supplies expected outputs

Original R2E extracts function dependencies, generates inputs/harnesses with GPT-4-Turbo and obtains expected outputs by executing the original code. Execution and coverage feedback refine tests. R2E-Eval1 has 246 tasks from 137 repositories. It is principally function reconstruction and observational equivalence, with some manual setup. MIT program-analysis, test-generation and execution code exists. This is valuable for refactoring/performance tasks, but the original implementation cannot itself define an unimplemented feature's desired semantics. ICML 2024 paper, implementation.

SWE-Mirror - transfer an issue family into a working repository

An issue is abstracted and transferred into a related, prepared repository. GPT-4.1 powers separate test and mirror agents: tests pass on original target code, then a mutation introduces the intended failure. Reversing it supplies the reference. The paper reports 60,671 tasks from 40 repositories across four languages, using three-state validation and hidden tests. No matching official generator was located. The exact-name KokerZhou repository is an unrelated SWE-bench Pro image mirror. This is issue transfer, not faithful replay of the source PR. Paper, September 2025.

DeNovoSWE and SWE-Future - broader reconstruction and forecasting

DeNovoSWE reconstructs whole repositories from capability documents aligned through runtime test profiling and draft/critic/repair. GPT-5.4/5.5 author/judge modules and a DeepSeek-V4-Pro-High solver play distinct roles. The paper reports 4,818 tasks; public HF metadata showed 3,675 rows. The repository is release material rather than generator code. Paper, project, data.

SWE-Future forecasts task families from a frozen history window and synthesizes tasks against a defined snapshot. It reports 200 tasks from 61 repositories with independently constructed hidden tests/reference patches. A corresponding official release was not located. It suggests a research direction for realistic future work, rather than a near-term bootstrap replacement. Paper, June 2026.

5. Constructing terminal environments

TerminalWorld - reconstruct real terminal recordings

EuniAI's TerminalWorld reports 80,870 recordings, 9,492 filtered recordings, 5,035 reproduced environments and 1,530 validated tasks; 200 receive human verification. Instructions express outcomes, dependencies are reconstructed and state-based tests are challenged with full, no-op and partial solutions in fresh containers. Apache-2.0 generation source is public. This supplies unusually useful verifier-discrimination checks and honest construction denominators. Agent SDK model configuration is exposed rather than proving one fixed author model across all runs. Paper, May 2026, implementation.

Terminal-World - skill-derived, not the recording project

A different paper composes skills, skill graphs and personas into specifications. DeepSeek-V3.2 generates tasks/trajectories; Gemini3-Flash builds environments. It reports 6,884 accepted specifications yielding 5,723 environments, or 83.1% construction success after specification filtering. Training is SFT. No matching official generator was located. Pointing this paper to EuniAI/TerminalWorld would be a mistaken identity. Paper, May 2026.

Endless Terminals - accessible staged synthesis

Qwen3-32B creates task templates, initial-state tests, completion tests and Apptainer definitions, followed by execution and solution sampling. The project publishes 3,255 tasks with actual Apache-2.0 generation/training code, Harbor conversion and reward-file tooling. It is a useful baseline for open-teacher staged generation. Initial-state validity, existence of a solution and downstream usefulness should still be measured separately. Paper, January 2026, repository, dataset.

SETA - synthesis and configurable evolution are public

SETA-Synth standardizes heterogeneous source material; SETA-Evol changes difficulty and context. The paper reports over 4,500 environments. Apache-2.0 source includes synthesis/evolution modules, configurations, worker orchestration, upload and training integration. Inspected evolution documentation specifies one strategy per run with explicit chaining; it should not automatically be described as a learned scheduler. Remote execution and Daytona integration make it a practical comparator. Paper, July 2026, implementation.

CLI-Gym - corrupt environments, not just code

CLI-Gym starts from healthy SWE-smith images and disrupts their environments to create recovery tasks checked by existing tests. It reports 1,655 tasks from 29 repositories. The MIT generator writes task metadata, Docker/Compose configuration and selected-test runners. The inspected output uses the earlier Terminal-Bench format, so compatibility should be established through an adapter. Useful when the intended skill is dependency, configuration or system diagnosis. Paper, February 2026, repository.

TMax - a real generator with a different quality policy

TMax composes domains, skills, personas, fixtures and difficulty atop shared base images. It releases actual generation/training code and about 14,600 tasks, with Harbor export and Daytona evaluation. The paper uses Gemini3-Pro; current README configuration names Gemini3.1-Pro-preview. Crucially, the method deliberately skips teacher/oracle correctness validation and relies on RL filtering of zero-pass tasks. Its executability checks should not be inherited as Tasksmith's acceptance bar. Paper, June 2026, generation code.

SkillSynth - oracle success and RL quality diverge

Human-written skills become scenario graphs; sampled paths guide planner and constructor agents, followed by execution, rubric review and repair. Of 3,721 candidates, 3,560 pass the oracle, but 3,423 pass both oracle and rubric. The 137 rubric failures are retained for SFT and explicitly excluded from RL. The paper reports 721 recovered tasks and $27.3 per verified instance. No author-linked generator was located. This is direct evidence for separating execution validity from RL acceptance. Paper, April 2026.

Meta-Task - make environment authoring a terminal task

An agent works in Docker to construct task artifacts, execute its oracle/tests and repair problems. Qwen3.5-397B-A17B is author, solver and judge; Claude Code scaffolds authorship, Terminus-2 samples solutions and Harbor schedules work. Approximately 15,000 packages lead to 14,040 attempted trajectories, 5,004 passing trajectories and 3,221 after judge filtering. No author-linked implementation was located. This is very close to Tasksmith's intended authoring architecture, but role separation here does not mean different model weights. Paper, July 2026.

CLI-Universe - independent tests and solutions

Capability taxonomy and research produce a blueprint; agents materialize assets and Docker environments, then separately author tests and solutions without seeing each other's outputs. Hint-conditional and fail-to-pass filters serve different purposes: difficulty and state-change validation. Five filters retain 33.6% of candidates; the training set contains 6,000 trajectories. No matching official generator/data release was located. This is a particularly relevant construction contract for reducing author/verifier shared assumptions. Paper, June 2026.

TermiGen, TerminalTraj and Nemotron-Terminal

TermiGen uses taxonomy-driven proposals, feasibility checks and iterative Docker/test construction; it subsequently adds controlled trajectory errors for recovery training. It reports over 3,500 environments. The official repository primarily releases task artifacts and BashAgent runtimes, not the authoring pipeline; no root license was found. Paper, release.

TerminalTraj reports 32,325 Docker environments and 50,733 verified trajectories. Its current release includes data/models and a 5,660-instance environment subset; the inspected six-file repository does not include the generator. The environment release makes older "no environments available" descriptions outdated. Paper, repository.

NVIDIA Terminal-Task-Gen combines adapted datasets with skill/seed-driven terminal synthesis. DeepSeek-V3.2 generates tasks and trajectories; nine domain images reduce setup overhead. Synthetic tasks and the Nemotron-Terminal corpus are released, but the exact authoring pipeline was not located. Findings about the SFT value of failed trajectories do not justify invalid RL rewards. Paper, tasks, corpus.

Terminal-Universe and NexForge - additional input routes

Terminal-Universe reconstructs pre-action workspaces from trajectory file operations, completes missing dependencies and generates new tasks/feedback. It reports 68,263 reconstructions, 40,194 assessed environments and 37,273 judged sufficient environments. Sufficiency is partly judge-based; reconstruction fidelity is a separate uncertainty. An official generator was not located. This could eventually use existing agent traces, but is not proof that inferred files are historically accurate. Paper, September 2026.

NexForge starts from practical requirements, controls scenario distribution and assembles executable terminal/office workspaces. It reports 43,200 terminal tasks. The project publishes research/model material; a corresponding generator was not located. It broadens task ideation beyond repository mutation. Paper, July 2026, project.

6. Adaptive generation and self-play

TaskPilot / FrogNano - adapt to the current coding learner

TaskPilot authors problem statements, reference patches and hidden F2P tests from SWE-rebench snapshots. It checks original failure, reference success and P2P stability, then uses current 4B learner rollouts to revise and calibrate tasks. Five roughly 300-task rounds are reported. Generator identities are undisclosed; a stronger generator is used in the revised fifth round. The generator is distinct from, and not jointly trained with, the learner. Leaf is the solver harness. Paper, September 2026.

No public TaskPilot generator was located; Microsoft's debug-gym is separate infrastructure. The actionable idea is diagnosis before regeneration: revise ambiguity, verifier unfairness or difficulty according to actual traces, while revalidating the resulting task. Official project.

CalibForge - behavioral calibration across solvers

DeepSeek-V4-Pro authors and self-solves candidates. Multi-solver calibration uses V4-Flash, GLM-5 and Kimi K2.5; contrastive calibration seeks strong-pass/weak-fail outcomes. Full traces can trigger revisions to any task component. Up to 50 calibration rounds are allowed. Reported acceptance rises from 19% to 96% among candidates already structurally validated and self-solved; this is not raw-seed yield. Training is SFT. Paper, August 2026.

The project releases 5,431 Harbor tasks, images and trajectories. Its inspected repository has only three files; no generator or explicit root license was found. Project, tasks.

Environment Evolution for Terminal Agents - the newly supplied paper

Tencent's method evolves accepted tasks along **scenario novelty, skill rarity and execution length**. A proposer/reviewer refines plans; a modifier changes artifacts; oracle, no-op and rubric checks gate acceptance. Development includes human review. The comparison fixes Claude Opus 5 as synthesizer and trains Qwen3.6 learners; same-model construction/learning is future work. Paper, September 2026.

Generation is off-policy, but seed selection uses solver performance and the lineage scheduler advances using learner success. The 47,678-to-127 seed reduction combines quality and difficulty filters; it is not a measured defect rate. "Intrinsic difficulty" remains an estimated reference-distribution quantity, not a universal correctness oracle. No official generator was located.

For Tasksmith, this motivates optional task lineages after faithful PR conversion. Method, experiments and version note.

Recursive Synthesis / RST - extend solutions, realign the contract

DeepSeek-V4-Pro recursively extends executable solutions, then realigns the environment, verifier and instruction. Forty operators, lineage/domain caps, bounded repair and fresh Harbor/Daytona validation produce 37,484 tasks from 639 seeds. Reported yield is 498-572 accepted tasks per 1,000 seed attempts, distinct from later-stage pass rates. Task artifacts and prompts are available; generator code was not located. Revalidating public-contract alignment after every extension is the key lesson. Paper, August 2026, dataset.

Envs-FORGE - controlled transformations and selection

Seed pass-rate estimates, fixed transfer priors and a per-seed MILP select depth/breadth transformations that increase, reduce or diversify difficulty. The experiment reaches 100 accepted tasks after 291 attempts and 2.881 million synthesis tokens. Generator identity is not specified. Paper, August 2026.

The linked toolkit's non-default SynAgenticData branch contains coherent Harbor task synthesis and smoke validation. Inspected code implements few/self/evol modes, but the paper's MILP/frontier selector was not found. This is a partial related implementation, not "no code" and not a verified complete reproduction. Relevant branch.

Self-play SWE-RL / SSR - train the bug injector and repairer

Meta's SSR uses one shared trainable policy as injector and solver. It assumes prepared repository images, constructs bug/test-weakening artifacts and uses validity checks including inverse mutation tests per modified file. Generator rewards penalize invalid, universally solved and universally failed tasks. The solver's formal specification differs from a human issue description. No official SSR generator was located. This supplies useful verifier challenges, but does not solve repository bootstrap or map directly onto a fixed API-model workflow. Paper, December 2025.

Socratic-SWE - optimize generation using learning signals

A shared Qwen3.5-9B generator/solver is trained, while Qwen3.6-27B extracts skills from traces. Validation checks format, grounding, stability and semantics. Generator reward compares candidate-induced solver gradients with gradients from trusted BeyondSWE tasks. The paper reports three rounds of 12,000 validated instances without a raw candidate denominator. No official implementation was located. Skill registries are immediately reusable as an idea; gradient-alignment optimization requires learner access beyond opaque Sonnet/Opus APIs. Paper, June 2026.

SPADE - executable environment self-play with public source

One trainable policy designs Python environments and acts in them. Hint-based regret compares performance with and without privileged help; environment memory and grounding support diversity. Main domains are cognitive games and tool use, not repository repair. MIT source includes generation, validation and multiple training paths. This demonstrates actual joint designer/solver learning rather than simply alternating prompts to a frozen API model. Paper, August 2026, implementation.

BigBang - learn whether the quality proxy predicts learning

The proposed system has a generator modifying synthesis programs, a critic assessing artifacts and a meta-critic comparing assessments with downstream training outcomes. The inspected Apache-2.0 BigBang-v1 repository contains evaluation, tools and data-download material, not the generator/critic/meta-critic synthesis system. Its relevance is conceptual: eventually validate our quality rubric against actual learning gains. Official method, release.

7. Adjacent work and important boundaries

Function-level reasoning and test generation

Absolute Zero Reasoner jointly trains a proposer and solver around code-execution-validated deduction, abduction and induction. MIT generation/training source is public. "Zero data" refers to external post-training examples, not absence of pretraining. These are code reasoning problems rather than arbitrary repository repair environments. Paper, code.

SCALER converts programming problems into parameterized environments with deterministic oracles, difficulty controls and randomized instances; adaptive selection maintains useful learning signals. Its public Apache-2.0 implementation has real synthesis/controller code, with paths/configuration needing adaptation. The reported pool has 2,739 environments from 4,973 problems. Useful for scalable input variation, not dependency reconstruction. Paper, implementation.

AceCoder generates and filters tests, then uses them for reward modeling/RL. Its MIT implementation contains real question/test-generation code and AceCode data. In the inspected OSS path, authoring receives reference programs, so reference-correlated tests remain a concern. It is primarily function-level coding data. Paper, code.

SYNTHETIC-1 releases two million reasoning responses, not two million environments. Genesys provides generation and verification components. Coding problems can use execution tests, while its real-world single-file SWE tasks use an LLM judge against post-commit content. The verification modes should not be collapsed. Release methodology, Genesys.

OpenCodeReasoning is competitive-programming trajectory distillation with execution filtering; NVIDIA-NeMo Skills exposes its generation workflow. CodeRL is an earlier actor/critic coding-RL system using programming problems and tests. Both are relevant training infrastructure, rather than general PR-to-environment generators. OpenCodeReasoning paper, generation documentation, CodeRL.

Trajectories or patch rewards are not behavioral environment validation

SERA's Soft-Verified Generation synthesizes a change, turns it into a description and asks a second rollout to reproduce it. Apache-2.0 generation/training code exists. The inspected filter measures changed-line recall; even a threshold of one is not exact patch equivalence and does not penalize arbitrary extra changes. This is a valuable SFT approach without mandatory behavioral execution. Paper, implementation, filter.

SWE-RL uses patch-sequence similarity for its central reward. The public reward implementation calls SequenceMatcher rather than running a test oracle. It should not be counted as a test-verifiable environment generator. SWE-Fuse uses SWE-smith environments and expands/filter trajectories before RL; its paper contains placeholder release links. SWE-RL paper, reward source, SWE-Fuse paper.

Machine learning engineering and security

MLE-Smith turns datasets into competition-style ML tasks through brainstorming, design, repair and execution in MLE-Dojo. GPT-5 powers all authoring agents. It samples 300 datasets, proposes 807 tasks and retains 606 verified tasks across 224 datasets. A corresponding official generator was not located. Its relevance is resource-aware metric tasks: data leakage, split integrity and performance variance need domain-specific verification. Paper, October 2025.

SEC-bench and SecVerifier provide actual MIT collection, preprocessing, build/evaluation and builder/exploiter/fixer/reproducer code for reproducible vulnerability tasks. They show that specialist verification can require more than generic unit tests. SEC-bench-Pro extends to V8, SpiderMonkey and Linux; its Linux execution requirements include KVM. These are useful resource-contract references, rather than something to silently force into a CPU-only generic container. Paper, SEC-bench, SecVerifier, Pro.

Synthetic tool worlds and runtime adapters

Agent World Model synthesizes scenarios, tasks, SQLite state, APIs, MCP servers and verifiers. Its public generator has both pure-code and code-augmented LLM-judge modes; 1,000 worlds have nominally ten tasks each. No root license was found in the inspected repository. This is a useful explicit-state architecture, but synthetic APIs bypass much of real-repository build complexity. Paper, source.

Harbor Adapters ports over 80 benchmarks and reports parity checks; Harbor-Index curates 82 tasks spanning 29 benchmarks through filtering, AI/human audit and repair. This is adaptation and quality infrastructure rather than task invention. Benchmark adaptation itself needs evidence that reward semantics survive conversion. Paper, September 2026, Harbor repository.

8. What strong verification actually requires

Human task design remains a useful constraint

Ivan Bercovich's Good Benchmarks is a practitioner essay drawn from Terminal-Bench contributors, not a systematic quantitative survey. It argues for realistic outcomes, sufficient instructions, solvability within resources and tests that accept alternative correct approaches. It also explicitly argues for human authorship/review and warns that closed author/judge loops can optimize toward artificial tasks. Fully automated expert quality is therefore a hypothesis to test, not a conclusion of the paper. The earlier guideline is a separate publication. Good Benchmarks, earlier guideline.

Challenge the grader, then preserve legitimate solutions

SWE-Mutation releases agentic mutants intended to test whether verification systems distinguish plausible incorrect code. Its dataset contains 2,636 mutants from 800 original instances; actual MIT mutation/evaluation code exists. For Tasksmith, mutation score should be recorded alongside functional coverage, with equivalent or irrelevant mutants reviewed separately. Paper, implementation.

Hardening Agent Benchmarks with Adversarial Hacker-Fixer Loops audits 1,968 tasks and identifies 323 hackable cases. A hacker seeks unearned reward, a fixer repairs the verifier, and a legitimate solver checks that repairs preserve solvability. The Apache-2.0 harden-v0 implementation operates on Harbor tasks and includes replay and shared defenses. "No hack found within budget" is evidence, not proof of universal robustness. Paper, source.

TerminalWrench's separate release reports 331 hackable environments, 3,632 hack traces and 2,352 legitimate baselines. Do not merge that count with the later paper's 323/1,968 audit denominator. Dataset paper, release.

Independent audits change how scale claims should be read

Terminal-Bench 2.1 fixed 28 of 89 tasks, including external-dependency drift and inadequate resources. Versioned source alone does not pin mutable external systems. Maintainer release notes.

River audits TMax using eight rubrics and solver filtering, retaining River-TMax-3.5K. Its authors label over 60% of audited TMax environments defective, while acknowledging audit false positives/negatives. This is an author-reported audit, not independently established ground truth. It nevertheless challenges the assumption that zero-pass RL filtering handles all quality problems. River is curation/training, not a new generator; a corresponding standalone release was not located. Paper, official site.

Proposed Tasksmith evidence contract

Use pass/fail gates for correctness and a structured scorecard for confidence. Do not let a high average score compensate for a fatal leak or broken reference. The following is a proposed acceptance policy, not a claim that any cited project implements all of it.

- **Instruction:** each requirement is observable, with a stated source and verifier mapping. Public interface requirements are explicit; implementation choices remain open.

- **Execution:** original regressions, new behavior and gold solution have identifiable test results. Fresh replay is stable under the declared resources; setup failures are classified separately.
- **Sensitivity:** no-op, meaningful partial fixes and plausible wrong implementations fail for the expected reasons.
- **Fairness:** independent correct implementations, when available, pass. A mismatch triggers contract review rather than forced agreement with the reference.
- **Isolation:** learner-visible files, image layers, Git objects, logs, caches and network access are inspected for answers or grading access. Trusted grading is performed on controlled artifacts.
- **Rollouts:** successes are classified as legitimate or suspicious; failures as learner, specification, verifier or infrastructure failures, with evidence references and uncertainty.
- **Human calibration:** audit a stratified sample and use discovered mistakes to improve automated checks. Keep automated and human-verified release labels distinct.

LLM reviewers are useful for ambiguity, trace interpretation and suspected shortcuts. Deterministic executable checks should supply the reward wherever the task admits them. If a judge-based reward is necessary, publish its model/prompt/version, measure agreement and stability, and label the reward type.

9. Models, yield and cost: compare the right objects

Role separation is not model independence

Several designs use one model in multiple roles. SWE-Synth uses Qwen2.5-Coder32B for mutants and repairs; Meta-Task uses Qwen3.5-397B-A17B for authorship, solving and judging. SWE-smith uses a different bug writer and repair teacher. CLI-Universe separates the information visible to test and solution agents. TaskPilot's generator is distinct from its learner but not named. SPADE and SSR train a shared policy in both roles. The project-specific sources in chapters 3-7 support these distinctions.

For our first system, I recommend independent contexts with explicit information boundaries: a PR analyst may see the diff; a verifier designer first works from the behavioral contract; an oracle builder receives the reference; a solver sees only the public task; an auditor can inspect everything after execution. Model diversity is a useful experimental variable, not a correctness guarantee. Comparing Pi and OpenCode should hold model, task revision and budget constant so harness effects are not confused with model effects.

Published funnels are not interchangeable yield estimates

Work	Reported funnel or rate	What its denominator means
SWE-rebench V2	583,809 -> 41,349 -> 32,079	Eligible PRs -> executable F2P tasks -> clear instructions
SWE-smith	50.1%	Candidate mutations that break previously passing tests
SWE-Next	102,582 -> 2,308	Candidate commit pairs -> retained instances
TerminalWorld	9,492 -> 5,035 -> 1,530	Filtered recordings -> reproduced environments -> validated tasks
SkillSynth	3,560 vs 3,423 of 3,721	Oracle success versus oracle plus rubric acceptance
CalibForge	19% -> 96%	Calibration among already validated, self-solved candidates
Envs-FORGE	291 attempts -> 100 tasks	Repeated synthesis attempts, not 100% first-attempt success
SWE-Synth	21,804 -> 9,459 -> 3,018	Mutants -> verifiable bugs -> successful repair trajectories

Numbers above refer to the cited paper/release versions in the corresponding entries. They do not establish a ranking of generator quality. Requirements, repository selection, retries, human work and acceptance checks vary.

Track our own funnel from received PRs through eligible scope, reusable bootstrap, gold execution, verifier challenges, rollout audit and release. Keep the original denominator visible. Track recovery yield separately: a dependency failure repaired on attempt two is useful progress, not a new PR or a discarded first attempt. A task that passes quality checks but is too easy belongs in the accepted catalog with difficulty metadata.

Costs also need a common boundary. Record authoring tokens, review tokens, solver tokens, build CPU/GPU time, artifact storage, transfers, failed attempts and any human work. Divide by final accepted tasks only after including rejected attempts. Keep charged usage separate from reservations. Reuse a healthy environment when instructions or tests change; rerun only the validation whose inputs changed, with a final fresh acceptance replay.

10. How I would implement Tasksmith next

This is a proposed next implementation, not work performed during this research. The goal is **the best faithful environment for each eligible PR, with explicit evidence when the current resource or verification contract cannot support it**. Evolved tasks are additional products linked to their seed.

Stage	Concrete output	Validation before advancing
Understand the PR	Behavioral delta, interfaces, dependencies, resource profile, candidate task framings	Separate intended behavior from incidental diff changes; identify missing evidence
Bootstrap	Reusable dependency recipe, immutable image digest, rebuild/test commands and parser	Fresh base/gold execution; collect real tests; classify pre-existing failures
Choose a task	Concise public instruction and requirement-to-check map	Sufficient observable contract; no answer localization or invented obligations
Build grading	Protected oracle, hidden behavioral tests, regression scope	Gold passes; base lacks target behavior; partial and wrong fixes rejected
Audit isolation	Learner snapshot, artifact allowlist, grading boundary	Inspect history/layers/caches; verify network behavior and trusted reward path
Probe and repair	Versioned Sonnet/Opus traces, outcome classification, targeted edits	Distinguish agent failure from task failure; rerun affected evidence after repair
Release	Harbor bundle, provenance and quality evidence	Fresh final replay, version hashes, resource declarations and license metadata
Evolve optionally	Child tasks, mutation intent, independent evidence and difficulty estimates	Preserve seed; recheck each child; split data by lineage/repository family

Separate repository readiness from PR semantics

Use a provider-neutral bootstrap record, with remote Modal/Daytona implementations. Keep dependency cache identity tied to repository/runtime/platform, lockfiles, system dependencies, build flags and relevant environment assumptions. Repository-quarter or adjacent-commit reuse is a candidate-selection heuristic; verify compatibility before reuse. Record the source revision separately so every PR does not force a dependency-image rebuild. RepoLaunch, SWE-Next and SWE-rebench provide the main source precedents.

Measure three states explicitly: base with its original tests; base with the new grading tests; gold with those tests. A missing new API can be an intended base failure. A missing pytest plugin cannot. Feature-specific import/build errors therefore need causal classification, not either an unconditional rejection or an unconditional F2P label. Test discovery and structured result parsing are part of the evidence.

Keep orchestration explicit and authoring flexible

LangGraph can own stage contracts, checkpointing, budgets and repair routing. Pi/OpenCode can perform open-ended exploration and edits inside the authoring sandbox. They should return artifacts and execution evidence, while acceptance remains under the orchestrator's control. A verifier revision should resume verifier validation; it should not reset successful bootstrap work. This is an architectural recommendation; the survey is not a Pi-versus-OpenCode runtime benchmark.

Current Harbor documentation supports separate verifier environments and explicit network policies, but defaults and provider capabilities matter. Shared grading is the default, and baseline networking defaults to public. Confirm the pinned Harbor version and chosen provider implement the needed contract. A hostname allowlist is not bucket-path authorization: stage required assets before the learner run or use a narrowly scoped delivery mechanism. Sanitizing Git and disabling retrieval still cannot prove absence of pretraining contamination. Current Harbor task/network/verifier contract.

Run a five-PR experiment that explains its own failures

Choose five eligible PRs spanning a small bug fix, feature addition, insufficient existing tests, a dependency-sensitive build and a domain-specific requirement. Include repeated repository use to test cache reuse. Freeze the input set and resource scope before running. For each, report the exact stage reached, evidence, repair history, token/compute cost and acceptance outcome. Do not substitute easier PRs silently.

First compare the existing Tasksmith path with a reused bootstrap/PR-packaging baseline. Then add independent verifier challenges and trace-directed repair. Only after those work should we test curriculum evolution. Small solver samples diagnose obvious failures; they do not precisely estimate difficulty. Increase probes when uncertainty affects a decision, and keep a frozen held-out set of challenge solutions that authoring never sees.

The differentiation worth building is a reproducible **quality evidence layer**: independently challenge grading, preserve alternative correct solutions, route repairs accurately, reuse expensive setup, and release the full construction history. A new orchestration wrapper alone would duplicate substantial public work.

11. Public implementation catalog

The following pins identify source snapshots inspected in this review. "Present" means relevant generation/construction code exists, not that we reproduced a successful run. Code licenses below describe the inspected repository scope; datasets and embedded upstream code may carry other terms.

Project / snapshot	Released component	Code status / license
SWE-gen 14e185f4	PR construction, validation, rollout classification	Present / Apache-2.0
RepoLaunch db3eb23a	Build/test agent and adjacent-commit reuse	Present / MIT
SWE-Next b55c0841	Miner, dependency profiles, execution/export	Present / Apache-2.0
SWE-rebench V2 c71902a8	Prompts, base images, build/eval scripts	Partial / MIT
SWE-Factory 760b1758	Multi-agent environment builder	Present / special noncommercial/commercial terms
OpenSWE c2294ce5	Multi-agent builder and distributed workers	Present / derived special terms
R2E-Gym 0d94c4eb	Repository-configured generation and evaluation	Present components / Apache-2.0
SWE-smith 9b74ac08	Mutators, profiles, validation, trajectories	Present / MIT
SWE-Synth 60efb824	Mutants and Docker validation	Present / no root license found
SWE-Flow 7da5b046	Tracing-based reconstruction	Present / MIT
THUDM SWE-Dev 72917b61	Test descriptions, generation and execution	Present / MIT
R2E bcbed156	Dependency slicing, test generation, execution	Present / MIT
TerminalWorld 784698ba	Recordings -> environment -> trial-validated tests	Present / Apache-2.0
SETA e4715b01	Synthesis/evolution/training	Present / Apache-2.0
Endless Terminals 99f4c74b	Staged generation and conversion	Present / Apache-2.0
CLI-Gym 48bb920b	Environment inversion	Present / MIT
TMax 7387d2f9	Compositional generator and training	Present / Apache-2.0

Project / snapshot	Released component	Code status / license
SPADE ebd40ec8	Joint environment designer/solver training	Present / MIT
Envs-FORGE linked branch 2a1d65ec	Harbor synthesis; selector unverified	Partial / Apache-2.0
harden-v0 342b8474	Hacker/fixer/solver/replay loop	Present / Apache-2.0
SWE-Mutation c9485dc1	Plausible wrong-patch generation/evaluation	Present / MIT
SERA 1ca8673b	Soft-verified trajectory synthesis	Present / Apache-2.0
Agent World Model 85e322f6	SQL/MCP world generation	Present / no root license found
CalibForge 4a219dcd	Release description/assets	Generator absent in inspected tree
BigBang-v1 5128884c	Evaluation/tools/downloads	Generator absent / Apache-2.0
TerminalTraj 01305cbf	Data/models and environment subset	Generator absent / Apache-2.0

Fastest paths to useful code

- SWE-gen: inspect `create/claude_code_runner.py`, `tools/validate.py` and `analyze/classifier.py` for construction, acceptance and outcome diagnosis.
- RepoLaunch: inspect `launch/scripts/adjacent_commit_run.py` and the memory documentation for amortized setup across commits.
- SWE-smith: inspect `swesmith/bug_gen/`, `harness/valid.py` and `profiles/` for mutation challenges and reusable test execution.
- THUDM SWE-Dev: inspect `swedev/testcases/` for description-to-test generation and reference/base evaluation.
- TerminalWorld: inspect `environment_building/` and `test_generation/refine_task.py` for execution-driven reconstruction and partial-solution checks.
- SETA: inspect `datasynth/evol_pipeline/` for strategy-driven task transformations and worker configuration.
- harden-v0: inspect `harden/loop.py`, `workspace.py` and `precheck_cache.py` for adversarial repair, replay and cached legitimate-solution checks.

These paths belong to the linked repository snapshots above; they were read rather than executed.

12. Scope, evidence and remaining uncertainty

This survey prioritizes repository-scale and terminal coding environments through 10 September 2026, with function-level reasoning, specialist tasks and training-data systems included to clarify boundaries. Discovery used the supplied papers, related-work references, targeted arXiv/GitHub/Hugging Face searches and official project links. Main repository trees, selected non-default branches, licenses, generation entrypoints, evaluator logic and dataset metadata were inspected where relevant. Consequential claims were checked against primary papers and source files.

This is a broad targeted landscape review, not a formally exhaustive systematic review. It does not audit every benchmark, general web/GUI environment, every synthetic instruction dataset or all self-play literature. "Not located" means no matching official release was found through the searches and links inspected; future releases, unlinked code and private implementations may exist. Some release counts came from HF metadata rather than downloading full datasets. Auto-gated and smaller public releases are labeled explicitly.

Paper counts, current repository counts and public dataset sizes are preserved as different claims. Same-name projects are disambiguated. RepoLaunch's reuse result, CalibForge's calibration result and synthetic mutation yields have different denominators. No source repository was executed, no Docker image was pulled and no paid rollout campaign was launched for this review. Runtime reliability, provider compatibility and actual quality of sampled tasks remain the next empirical checks.

The primary unresolved research question is not whether an LLM can emit a Harbor bundle. Public systems already do that. It is whether automated construction and review can reliably accept legitimate solutions, reject shortcuts, preserve realistic task intent and improve learners at an acceptable total cost. Repo2RLEnv should make those claims measurable and publish the evidence with each environment.